

Referat - E-shop de articole sportive bazat pe recomandări

Alex Ioan Moraru

17 mai 2026

1 Introducere

Aplicația reprezintă întocmai ideea unui magazin online bazat pe articole sportive. Utilizatorul poate căuta produse în voie, dar are și posibilitatea de a completa un chestionar pe baza căruia i se vor prezenta atât sporturi potrivite pentru acesta (sporturi ce i-ar putea plăcea), cât și produse ce i-ar fi utile (bazate atât pe sportul recomandat, cât și pe nivelul său sportiv).

Aplicația este construită pe trei mari module. Baza de date este construită prin intermediul PostgreSQL (implicit limbajul de scripting PL/pgSQL). Serverul este scris în Spring Boot și folosește JDBC pentru comunicarea cu baza de date. Frontend-ul este o aplicație scrisă prin intermediul librăriei React ce comunică cu serverul prin intermediul principiului REST.

2 Cerințele de business

- **Autentificare și Gestiunea Utilizatorului**
 - Utilizatorul se poate înregistra în aplicație cu email și username unice.
 - Utilizatorul poate modifica datele personale.
 - Doi utilizatori nu pot avea același username sau același email.

- **Sistemul de Chestionare și Recomandări**

- Utilizatorul poate completa formularul de mai multe ori, sistemul păstrând istoricul.
- Pentru fiecare răspuns dat la chestionar, sistemul generează recomandări de sporturi și produse, acesta păstrând istoricul.
- Sistemul poate avansa nivelul utilizatorului pentru un sport de la începător la intermediar sau avansat.
- Utilizatorul nu poate avea două nivele la același sport.
- Bugetul minim nu poate depăși bugetul maxim într-un chestionar.
- Un sport nu poate apărea de două ori în aceeași recomandare.
- Un produs nu poate apărea de două ori în aceeași recomandare.

- **Gestiunea Produselor și a Stocului**

- Utilizatorul poate căuta și vizualiza produse din magazin fără a completa chestionarul.
- Utilizatorul poate face operații de tip CRUD pentru produse.
- Utilizatorul poate modifica stocul produsului.
- Stocul unui produs nu poate fi negativ.

- **Comenzi și Istoric Tranzacțional**

- Utilizatorul poate crea comenzi de produse și să vadă istoricul acestora.
- Un produs șters din magazin rămâne vizibil în istoricul comenzilor și al recomandărilor anterioare.
- O comandă nu poate conține de mai multe ori același produs.
- O comandă reține prețul produsului la momentul achiziției, independent de modificările ulterioare ale prețului.

3 Normalizarea bazei de date

3.1 Definirea dependențelor funcționale

Se consideră următoarea relație:

$R[\text{user_id}, \text{username}, \text{email}, \text{password_hash}, \text{first_name}, \text{last_name}, \text{address}, \text{birth_date}, \text{phone}, \text{user_created_at}, \text{profile_id}, \text{occupation}, \text{goal}, \text{preferred_environment}, \text{daily_schedule}, \text{free_hours_week}, \text{activity_level}, \text{effort_tolerance}, \text{prefers_team}, \text{medical_notes}, \text{budget_min}, \text{budget_max}, \text{profile_updated_at}, \text{sport_id}, \text{sport_name}, \text{sport_description}, \text{is_team_sport}, \text{is_outdoor}, \text{effort_level}, \text{min_budget}, \text{sport_image_url}, \text{sport_is_active}, \text{criteria_id}, \text{criteria_key}, \text{criteria_value}, \text{weight}, \text{level_id}, \text{current_level}, \text{level_started_at}, \text{level_last_updated}, \text{session_id}, \text{session_created_at}, \text{user_level_at_time}, \text{rec_sport_id}, \text{compatibility_score}, \text{rank}, \text{category_id}, \text{category_name}, \text{parent_id}, \text{category_description}, \text{product_id}, \text{product_name}, \text{product_description}, \text{price}, \text{target_level}, \text{brand}, \text{product_image_url}, \text{product_is_active}, \text{product_created_at}, \text{stock_id}, \text{stock_quantity}, \text{stock_last_updated}, \text{order_id}, \text{order_status}, \text{total_amount}, \text{shipping_address}, \text{order_created_at}, \text{order_updated_at}, \text{order_item_id}, \text{order_item_quantity}, \text{unit_price}, \text{rec_product_id}, \text{reason}]$

Definim mulțimea Σ a dependențelor funcționale. Aceasta este formată din reuniunea următoarelor dependențe:

- 1. Users

- $\text{user_id} \rightarrow \text{username}, \text{email}, \text{password_hash}, \text{first_name}, \text{last_name}, \text{address}, \text{birth_date}, \text{phone}, \text{user_created_at}$
- $\text{username} \rightarrow \text{user_id}$
- $\text{email} \rightarrow \text{user_id}$

- 2. User Profiles

- $\text{profile_id} \rightarrow \text{user_id}, \text{occupation}, \text{goal}, \text{preferred_environment}, \text{daily_schedule}, \text{free_hours_week}, \text{effort_tolerance}, \text{prefers_team}, \text{medical_notes}, \text{budget_min}, \text{budget_max}$

- **3. Sports**

- `sport_id` → `sport_name`, `sport_description`, `is_team_sport`, `is_outdoor`, `effort_level`, `min.budget`, `sport_image_url`, `sport_is_active`
- `sport_name` → `sport_id`

- **4. Categories**

- `category_id` → `category_name`, `parent_id`, `category_description`
- `category_name` → `category_id`

- **5. Products**

- `product_id` → `product_name`, `product_description`, `price`, `category_id`, `sport_id`, `target_level`, `brand`, `product_image_url`, `product_is_active`, `product_created_at`

- **6. User Sport Levels**

- `level_id` → `user_id`, `sport_id`, `current_level`, `level_started_at`, `level_last_updated`

- **7. Sport Criteria**

- `criteria_id` → `sport_id`, `criteria_key`, `criteria_value`, `weight`

- **8. Recommendation Sessions**

- `session_id` → `user_id`, `profile_id`, `session_created_at`, `user_level_at_time`

- **9. Recommendation Sports**

- `rec_sport_id` → `session_id`, `sport_id`, `compatibility_score`, `rank`

- **10. Recommendation Products**

- `rec_product_id` → `session_id`, `product_id`, `sport_id`, `reason`

- **11. Products and Stock**

- $\text{stock_id} \rightarrow \text{product_id}, \text{stock_quantity}, \text{stock_last_updated}$
- $\text{product_id} \rightarrow \text{stock_id}, \text{stock_quantity}, \text{stock_last_updated}$

- **12. Orders and Order Items**

- $\text{order_id} \rightarrow \text{user_id}, \text{session_id}, \text{order_status}, \text{total_amount}, \text{shipping_address}, \text{order_created_at}, \text{order_updated_at}$
- $\text{order_item_id} \rightarrow \text{order_id}, \text{product_id}, \text{order_item_quantity}, \text{unit_price}$

3.2 Găsirea cheilor candidat

Definim matematic cele trei mulțimi pe baza locului în care atributele apar în dependențele din Σ :

- **Mulțimea S (Stânga):** Conține atributele care apar *numai* în partea stângă a cel puțin unei dependențe din Σ :

$$S = \{ a \in U \mid \exists (X \rightarrow Y) \in \Sigma \text{ a.î. } a \in X \wedge \nexists (U' \rightarrow V) \in \Sigma \text{ a.î. } a \in V \}$$

$$S = \{ \text{level_id}, \text{criteria_id}, \text{rec_sport_id}, \text{rec_product_id}, \text{order_item_id} \}$$

- **Mulțimea M (Mijloc):** Conține atributele care apar *și* în partea stângă, *și* în partea dreaptă a dependențelor din Σ :

$$M = \{ a \in U \mid \exists (X \rightarrow Y) \in \Sigma \text{ a.î. } a \in X \wedge \exists (U' \rightarrow V) \in \Sigma \text{ a.î. } a \in V \}$$

$$M = \{ \text{user_id}, \text{username}, \text{email}, \text{profile_id}, \text{sport_id}, \text{sport_name}, \text{category_id}, \text{category_name}, \text{product_id}, \text{session_id}, \text{stock_id}, \text{order_id} \}$$

- **Mulțimea D (Dreapta):** Conține atributele care apar *numai* în

partea dreaptă a dependențelor din Σ :

$$D = \{ a \in U \mid \nexists (X \rightarrow Y) \in \Sigma \text{ a.î. } a \in X \wedge \exists (U' \rightarrow V) \in \Sigma \text{ a.î. } a \in V \}$$

$D = \{ \text{password_hash, first_name, last_name, address, birth_date, phone,}$
 $\text{user_created_at, occupation, goal, preferred_environment, daily_schedule,}$
 $\text{free_hours_week, activity_level, effort_tolerance, prefers_team, medical_not}$
 $\text{budget_min, budget_max, profile_updated_at, sport_description, is_team_sport}$
 $\text{is_outdoor, effort_level, min_budget, sport_image_url, sport_is_active,}$
 $\text{parent_id, category_description, product_name, product_description, price,}$
 $\text{target_level, brand, product_image_url, product_is_active, product_created_a}$
 $\text{stock_quantity, stock_last_updated, criteria_key, criteria_value, weight,}$
 $\text{current_level, level_started_at, level_last_updated, session_created_at,}$
 $\text{user_level_at_time, compatibility_score, rank, reason, order_status,}$
 $\text{total_amount, shipping_address, order_created_at, order_updated_at,}$
 $\text{order_item_quantity, unit_price} \}$

$S = \{ \text{level_id, criteria_id, rec_sport_id, rec_product_id, order_item_id} \}.$

Este cheia candidat deoarece din aceste atribute le putem determina pe toate celelalte:

- $\text{order_item_id} \rightarrow \text{order_id}$
- $\text{order_id} \rightarrow \text{user_id, session_id}$
- $\text{user_id} \rightarrow$ toate atributele legate de utilizator
- $\text{rec_product_id} \rightarrow \text{product_id}$
- $\text{product_id} \rightarrow$ toate atributele legate de categorie, sport și produs

3.3 Descompunerea în BCNF (Boyce-Codd Normal Form)

O schemă de relație este în BCNF dacă pentru orice dependență funcțională netrivială $X \rightarrow A \in \Sigma^+$, X este supercheie. Deoarece relația noastră univer-

sală R are numeroase dependențe unde determinantul nu este supercheie, R nu se află în BCNF.

Pentru a normaliza schema, vom aplica riguros **algoritmul de descompunere în join fără pierdere de tip BCNF**, iterând prin dependențele din mulțimea Σ .

Pasul 1 (Inițializare): Fie relația de start $R_0 = R$ (conținând absolut toate atributele) și $\rho = \{R_0\}$.

Pasul 2 (Iterația 1 - Entitatea Users): Alegem dependența $X \rightarrow Y$ din Σ care încalcă BCNF:

$\text{user_id} \rightarrow \{\text{username}, \text{email}, \text{password_hash}, \text{first_name}, \text{last_name},$
 $\text{address}, \text{birth_date}, \text{phone}, \text{user_created_at}\}$

Deoarece user_id nu este supercheie în R_0 , descompunem R_0 în:

- $R_1 = X \cup Y = \{\underline{\text{user_id}}, \text{username}, \text{email}, \text{password_hash}, \text{first_name}, \text{last_name}, \text{address}\}$
(Aflată în BCNF).
- $R^{(1)} = R_0 \setminus Y$ (Păstrăm restul atributelor, user_id rămâne ca legătură).

$\rho = \{R_1, R^{(1)}\}$.

Pasul 3 (Iterația 2 - Entitatea User Profiles): Din $R^{(1)}$ alegem dependența:

$\text{profile_id} \rightarrow \{\text{user_id}, \text{occupation}, \text{goal}, \text{preferred_environment},$
 $\text{daily_schedule}, \text{free_hours_week}, \text{activity_level}, \text{effort_tolerance},$
 $\text{prefers_team}, \text{medical_notes}, \text{budget_min}, \text{budget_max}, \text{profile_updated_at}\}$

Descompunem $R^{(1)}$ în:

- $R_2 = \{\underline{\text{profile_id}}, \text{user_id}, \text{occupation}, \text{goal}, \dots, \text{profile_updated_at}\}$
(Aflată în BCNF).
- $R^{(2)} = R^{(1)} \setminus Y$ (Atributele profilului sunt eliminate din relația mare).

$\rho = \{R_1, R_2, R^{(2)}\}$.

Pasul 4 (Iterația 3 - Entitatea Sports): Din $R^{(2)}$ alegem dependența:

$\text{sport_id} \rightarrow \{\text{sport_name}, \text{sport_description}, \text{is_team_sport},$
 $\text{is_outdoor}, \text{effort_level}, \text{min_budget}, \text{sport_image_url}, \text{sport_is_active}\}$

Descompunem $R^{(2)}$ în:

- $R_3 = \{\underline{\text{sport_id}}, \text{sport_name}, \text{sport_description}, \dots, \text{sport_is_active}\}$
- $R^{(3)} = R^{(2)} \setminus \{\text{sport_name}, \text{sport_description}, \dots\}$

$\rho = \{R_1, R_2, R_3, R^{(3)}\}$.

Pasul 5 (Iterația 4 - Entitatea Categories): Din $R^{(3)}$ alegem dependența:

$\text{category_id} \rightarrow \{\text{category_name}, \text{parent_id}, \text{category_description}\}$

Descompunem $R^{(3)}$ în:

- $R_4 = \{\underline{\text{category_id}}, \text{category_name}, \text{parent_id}, \text{category_description}\}$
- $R^{(4)} = R^{(3)} \setminus \{\text{category_name}, \text{parent_id}, \text{category_description}\}$

Pasul 6 (Iterația 5 - Entitatea Products): Din $R^{(4)}$ alegem dependența:

$\text{product_id} \rightarrow \{\text{product_name}, \text{product_description}, \text{price}, \text{category_id},$
 $\text{sport_id}, \text{target_level}, \text{brand}, \text{product_image_url}, \text{product_is_active}, \text{product_creat}$

Descompunem $R^{(4)}$ în:

- $R_5 = \{\underline{\text{product_id}}, \text{product_name}, \text{product_description}, \text{price}, \dots\}$
- $R^{(5)} = R^{(4)} \setminus \{\text{product_name}, \text{product_description}, \text{price}, \dots\}$

Pasul 7 (Iterația 6 - Entitatea User Sport Levels): Din $R^{(5)}$ alegem dependența:

$\text{level_id} \rightarrow \{\text{user_id}, \text{sport_id}, \text{current_level}, \text{level_started_at}, \text{level_last_updated}\}$

Descompunem $R^{(5)}$ în:

- $R_6 = \{\underline{\text{level_id}}, \text{user_id}, \text{sport_id}, \text{current_level}, \text{level_started_at}, \text{level_last_updated}\}$
- $R^{(6)} = R^{(5)} \setminus \{\text{current_level}, \text{level_started_at}, \text{level_last_updated}\}$
(Atenție: **user_id** și **sport_id** sunt păstrate și în restul relației deoarece fac parte din cheile altor asocieri).

Pasul 8 (Iterația 7 - Entitatea Sport Criteria): Din $R^{(6)}$ alegem dependența:

$\text{criteria_id} \rightarrow \{\text{sport_id}, \text{criteria_key}, \text{criteria_value}, \text{weight}\}$

Descompunem $R^{(6)}$ în:

- $R_7 = \{\underline{\text{criteria_id}}, \text{sport_id}, \text{criteria_key}, \text{criteria_value}, \text{weight}\}$
- $R^{(7)} = R^{(6)} \setminus \{\text{criteria_key}, \text{criteria_value}, \text{weight}\}$

Pasul 9 (Iterația 8 - Entitatea Recommendation Sessions): Din $R^{(7)}$ alegem dependența:

$\text{session_id} \rightarrow \{\text{user_id}, \text{profile_id}, \text{session_created_at}, \text{user_level_at_time}\}$

Descompunem $R^{(7)}$ în:

- $R_8 = \{\underline{\text{session_id}}, \text{user_id}, \text{profile_id}, \text{session_created_at}, \text{user_level_at_time}\}$
- $R^{(8)} = R^{(7)} \setminus \{\text{profile_id}, \text{session_created_at}, \text{user_level_at_time}\}$

Pasul 10 (Iterația 9 - Entitatea Recommendation Sports): Din $R^{(8)}$ alegem dependența:

$\text{rec_sport_id} \rightarrow \{\text{session_id}, \text{sport_id}, \text{compatibility_score}, \text{rank}\}$

Descompunem $R^{(8)}$ în:

- $R_9 = \{\underline{\text{rec_sport_id}}, \text{session_id}, \text{sport_id}, \text{compatibility_score}, \text{rank}\}$

- $R^{(9)} = R^{(8)} \setminus \{\text{rec_sport_id}, \text{compatibility_score}, \text{rank}\}$

Pasul 11 (Iterația 10 - Entitatea Recommendation Products):

Din $R^{(9)}$ alegem dependența:

$\text{rec_product_id} \rightarrow \{\text{session_id}, \text{product_id}, \text{sport_id}, \text{reason}\}$

Descompunem $R^{(9)}$ în:

- $R_{10} = \{\underline{\text{rec_product_id}}, \text{session_id}, \text{product_id}, \text{sport_id}, \text{reason}\}$
- $R^{(10)} = R^{(9)} \setminus \{\text{rec_product_id}, \text{reason}\}$

Pasul 12 (Iterația 11 - Entitatea Stock): Din $R^{(10)}$ alegem dependența:

$\text{stock_id} \rightarrow \{\text{product_id}, \text{stock_quantity}, \text{stock_last_updated}\}$

Descompunem $R^{(10)}$ în:

- $R_{11} = \{\underline{\text{stock_id}}, \text{product_id}, \text{stock_quantity}, \text{stock_last_updated}\}$
- $R^{(11)} = R^{(10)} \setminus \{\text{stock_id}, \text{stock_quantity}, \text{stock_last_updated}\}$

Pasul 13 (Iterația 12 - Entitatea Orders): Din $R^{(11)}$ alegem dependența:

$\text{order_id} \rightarrow \{\text{user_id}, \text{session_id}, \text{order_status}, \text{total_amount},$
 $\text{shipping_address}, \text{order_created_at}, \text{order_updated_at}\}$

Descompunem $R^{(11)}$ în:

- $R_{12} = \{\underline{\text{order_id}}, \text{user_id}, \text{session_id}, \text{order_status}, \text{total_amount}, \dots\}$
- $R^{(12)} = R^{(11)} \setminus \{\text{order_status}, \text{total_amount}, \text{shipping_address}, \text{order_created_at}, \dots\}$

Pasul 14 (Oprirea algoritmului): Relația rămasă $R^{(12)}$ conține acum exclusiv attributele:

$R^{(12)} = \{\text{order_item_id}, \text{order_id}, \text{product_id}, \text{order_item_quantity}, \text{unit_price}\}$

Observăm că acestea corespund exact ultimei dependențe din Σ :

$$\text{order_item_id} \rightarrow \{\text{order_id}, \text{product_id}, \text{order_item_quantity}, \text{unit_price}\}$$

Deoarece order_item_id determină toate atributele rămase, el este supercheie în $R^{(12)}$. Astfel, această ultimă relație este deja în BCNF și o vom nota direct cu R_{13} (entitatea **Order Items**).

Concluzie BCNF: În acest moment, nu mai există nicio dependență funcțională în mulțimea Σ care să încalce regula BCNF. Am demonstrat matematic că relația universală inițială R a fost descompusă *fără pierdere de informație* în setul final $\rho = \{R_1, R_2, R_3, \dots, R_{13}\}$. Fiecare schemă de relație din acest set respectă cu strictețe BCNF.

4 Diagrama Entitate-Asociere (E/A)



Figure 1: Diagrama E/A a bazei de date

4.1 Funcționalitatea Proiectului

4.1.1 Entități

- **users:** Utilizatori în aplicație. Pot primi recomandări, plasa comenzi, executa operații de tip CRUD pentru produse și pot modifica stocul produselor.
- **sports:** Sporturile din aplicație. Unui sport îi sunt asociate mai multe produse. Fiecare sport are criterii fixe pe baza cărora poate fi recomandat.
- **categories:** Entitate ce ajută la organizarea logică a produselor în magazin. Unei categorii îi sunt asociate mai multe produse.
- **products:** Un produs are asociat atât un singur sport, cât și o singură categorie. De asemenea, fiecărui produs îi este asociat un tabel de tip stock din necesitatea de a separa informațiile legate de produs de informațiile legate de numărul de produse disponibile.
- **stock:** Unui stoc îi este asociat doar un singur produs. Reține informații legate de disponibilitatea în magazin a produsului asociat.

4.1.2 Asocieri

- **user_profiles:** Asociere între utilizator și profilul său. Această tabelă conține informații despre răspunsurile la chestionar ale utilizatorilor. Între user și user_profile există o relație de one-to-many ce apare din necesitatea de a oferi utilizatorului un istoric al răspunsurilor sale (precum și un istoric al recomandărilor).
- **sport_criteria:** Fiecare sport are criterii pe care algoritmul de recomandări le folosește comparând preferințele utilizatorului cu proprietățile sportului. Astfel apare o relație de one-to-one între sport și sport_criteria din dorința de a separa informațiile de business ale sportului de criteriile de recomandare ale acestuia.
- **recommendation_sessions:** Asociere între user și user_profiles.

- **recommendation_sports:** Asociere între recommendation_sessions și sports. Reține sporturile recomandate în urma răspunsurilor la chestionar.
- **recommendation_products:** Asociere între recommendation_sessions și products. Reține produsele recomandate în urma răspunsurilor la chestionar.
- **orders:** Asociere între user și sesiunea de recomandări. ține evidența istoricului comenzilor făcute.
- **order_products:** Asociere între comenzi și produsele cumpărate. Reține informații despre prețul produsului la momentul respectiv și cantitatea cumpărată de utilizator.

4.2 Descrierea Entităților, Asocierilor și Cardinalităților

Arhitectura bazei de date este proiectată pentru a susține un ecosistem complex de e-commerce cu un motor de recomandare integrat. Relațiile dintre cele 13 tabele reflectă regulile stricte de business ale aplicației:

- **users – user_profiles (1:M):** user_profiles este tabela ce este responsabilă cu răspunsurile utilizatorului la chestionar. Utilizatorul poate actualiza răspunsul. Tabela va ține cont de istoric iar profilul în sine este reprezentat de cea mai nouă actualizare a tabelii.
- **users – user_sport_levels (1:M) – sports (1:M):** Tabela user_sport_levels în momentul în care utilizatorul completează chestionarul îi sunt asociate nivele pentru fiecare sport (cat de bine poate performa la sportul respectiv)
- **sports – sport_criteria (1:M):** Unui sport îi este asociată o tabelă de criterii fixe pe baza careia se fac recomandările.
- **categories – categories (1:M):** Relație pe baza careia se poate crea o structură arborescentă de categorii (subcategorii de produse).

- **sports / categories – products (1:M):** Un produs apartine unei singure categorii si maxim unui singur sport.
- **products – stock (1:1):** Relatie de tip one to one din necesitatea de a izola datele: informatiile despre produs si informatii despre disponibilitatea produsului
- **recommendation_sessions – recommendation_sports / recommendation_products (1:M):** O sesiune de recomandare generează liste multiple de sporturi compatibile și produse recomandate.
- **orders – order_items (1:M) – products (1:M):** Structura clasică de tranzacționare unde o comandă conține mai multe linii de detalii, fiecare linie indicând un produs unic și cantitatea aferentă.

4.3 Justificarea Constrângerilor și Alegerea Tipului de Date

4.3.1 Alegerea Cheilor Primare (PK) de tip UUID

Cheile primare sunt reprezentate prin intermediul atributelor id, de tip UUID in fiecare tabela. Aceasta decizie are rolul de a aduce proiectul cu un pas mai aproape de standardul industriei datorita securitatii asigurate de UUID. Alegerea cheii primare drept SERIAL este critica deoarece ii ofera unui atacator informatii despre id-ul de referinta a anumitor utilizatori / produse.

5 Scriptul de Creare și Populare a Tabelelor

Baza de date este construita prin intermediul unei arhitecturi de tip Domain Driven. Astfel, directoarele sunt organizate in 5 mari categorii, fiecare avand fisiere separate pentru tabele, functii, proceduri si triggere.

5.1 Users

```

-- Enum creat pentru a selecta modul in care cariera influenteaza ziua utilizatorului
CREATE TYPE daily_schedule_type AS ENUM (
    'FULL_TIME',
    'PART_TIME',
    'FLEXIBLE',
    'STUDENT',
    'RETIRED'
);

-- Enum creat pentru a retine cat de activ este utilizatorul
CREATE TYPE activity_level_type AS ENUM (
    'SEDENTARY',
    'LIGHT',
    'MODERATE',
    'ACTIVE',
    'VERY_ACTIVE'
);

-- Enum creat pentru a retine care este scopul pentru care utilizatorul doreste sa faca sport
CREATE TYPE goal_type AS ENUM (
    'WEIGHT_LOSS',
    'MUSCLE_GAIN',
    'CARDIO',
    'STRESS_RELIEF',
    'FLEXIBILITY'
);

-- Enum creat pentru a sti in ce mediu doreste utilizatorul sa faca sport
CREATE TYPE environment_type AS ENUM (
    'INDOOR',
    'OUTDOOR',
    'BOTH'
);

-- Enum creat pentru a sti cat efort poate tolera utilizatorul

```

```

CREATE TYPE effort_tolerance_type AS ENUM (
    'LOW',
    'MEDIUM',
    'HIGH'
);

-- Tabela de utilizatori
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- cheie primara
    -- Nu permitem la doi utilizatori sa aiba acelasi nume si/sau email => UNIQUE
    -- Nu permitem utilizatori fara nume sau email => NOT NULL
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(150) NOT NULL UNIQUE,

    -- Parola si numele sunt obligatorii
    password_hash VARCHAR(255) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,

    address TEXT,
    birth_date DATE NOT NULL,
    phone VARCHAR(20),
    created_at TIMESTAMP NOT NULL DEFAULT now()
);

-- Tabela pentru istoricul chestionarelor completate
CREATE TABLE user_profiles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- cheie primara

    -- Face legatura de tip one to many cu tabela users
    -- Daca utilizatorul este sters atunci se sterg toate raspunsurile
    -- pe care acesta le-a dat la chestionar
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    -- Toate variabilele pe baza caruia se construiesc recomandarile sunt NOT NULL

```



```

occupation VARCHAR(100),
goal goal_type NOT NULL,
preferred_environment environment_type NOT NULL,
daily_schedule daily_schedule_type NOT NULL,
free_hours_week INT NOT NULL CHECK (free_hours_week BETWEEN 0 AND 168),
activity_level activity_level_type NOT NULL,
effort_tolerance effort_tolerance_type NOT NULL,
prefers_team BOOLEAN NOT NULL,
medical_notes TEXT,

-- CHECK: nu poate exista un utilizator cu buget negativ
budget_min NUMERIC(10, 2) NOT NULL CHECK (budget_min >= 0),
budget_max NUMERIC(10, 2) NOT NULL CHECK (budget_max >= 0),
updated_at TIMESTAMP NOT NULL DEFAULT now(),

-- Constrangere logica: bugetul minim nu poate fi mai mare decat bugetul maxim
CONSTRAINT budget_range_valid CHECK (
    budget_max IS NULL OR budget_min IS NULL OR budget_max >= budget_min
);

```

5.2 Sports

```

-- Nivelul utilizatorului raportat la un anumit sport
CREATE TYPE current_level_type AS ENUM (
    'BEGINNER',
    'INTERMEDIATE',
    'ADVANCED'
);

CREATE TABLE sports (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(), --cheie primara

    -- Sportului ii necesita neaparat un nume unic => NOT NULL UNIQUE

```

```

name VARCHAR(100) NOT NULL UNIQUE,

-- Informatii despre sport
description TEXT,
is_team_sport BOOLEAN NOT NULL DEFAULT false,
is_outdoor BOOLEAN NOT NULL DEFAULT true,

-- CHECK necesar pentru formalizarea statisticilor sportului
effort_level INT NOT NULL CHECK (effort_level BETWEEN 1 AND 5),
min_budget NUMERIC(10, 2),
image_url VARCHAR(255),

-- Stergerea unui sport din baza de date se realizeaza prin setarea acestui
-- boolean la false (astfel ramane in istoric sportul cu toate dependentele sale
-- din baza de date, insa doar nu mai este disponibil pentru utilizatori)
is_active BOOLEAN NOT NULL DEFAULT true
);

CREATE TABLE user_sport_levels (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- cheie primara

    -- Asocierea one to many cu tabela users si sports
    -- Cand userul / sportul se sterge => se vor sterge si informatiile asociate
    -- din aceasta tabela
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    sport_id UUID NOT NULL REFERENCES sports(id) ON DELETE CASCADE,

    current_level current_level_type NOT NULL DEFAULT 'BEGINNER',
    started_at TIMESTAMP NOT NULL DEFAULT now(),
    last_updated TIMESTAMP NOT NULL DEFAULT now(),

    -- un utilizator nu poate avea doua nivele la acelasi sport
    UNIQUE (user_id, sport_id)
);

```

```

-- Maparea valorilor din tabela sports
CREATE TABLE sport_criteria (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- cheie primara
    sport_id UUID NOT NULL REFERENCES sports(id) ON DELETE CASCADE,

    -- Formeaza intrari de tip <cheie>: <valoare>
    -- Asemnator unui dictionar in care cheile sunt attributele tabelii sports
    criteria_key VARCHAR(50) NOT NULL,
    criteria_value VARCHAR(50) NOT NULL,

    weight NUMERIC(4, 2) NOT NULL DEFAULT 1.0,

    -- Pentru un anumit sport, o aceeaasi cheie nu poate avea doua attribute
    -- de tip valoare care sa fie diferite
    UNIQUE (sport_id, criteria_key, criteria_value)
);

```

5.3 Products

```

-- Tabela de categorii pentru produse
CREATE TABLE categories (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- cheie primara
    -- Categoria trebuie sa aiba neaparat un nume.
    -- Nu pot fi doua categorii care sa aiba acelasi nume
    name VARCHAR(100) NOT NULL UNIQUE,

    -- Atribut ce ajuta la formarea unei structuri arborescente de subcategorii.
    parent_id UUID REFERENCES categories(id) ON DELETE SET NULL,

    description TEXT
);

CREATE TABLE products (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- cheie primara

```

```

-- Numele produsului este obligatoriu
-- Pot exista mai multe produse cu acelasi nume (logic pentru un magazin)
name VARCHAR(200) NOT NULL,

-- Informatii despre produs
description TEXT,
price NUMERIC(10, 2) NOT NULL,

-- Asocierea de tip one to many cu tabela categories
category_id UUID NOT NULL REFERENCES categories(id),

-- Asociere de tip one to many cu tabela sports
-- ON DELETE SET NULL: pot exista produse care nu sunt asociate niciunui sport
sport_id UUID REFERENCES sports(id) ON DELETE SET NULL,

target_level VARCHAR(20) NOT NULL DEFAULT 'BEGINNER',
brand VARCHAR(100),
image_url VARCHAR(255),

-- Stergerea unui produs se face tot prin setarea booleanului la false
-- Astfel produsul ramane in istoricul comenzilor si al recomandarilor
is_active BOOLEAN NOT NULL DEFAULT true,

created_at TIMESTAMP NOT NULL DEFAULT now()
);

-- Tabela responsabila cu informatiile despre disponibilitatea unui produs in magazin
CREATE TABLE stock (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- cheie primara

    -- Asociere one to one cu tabela products
    product_id UUID NOT NULL UNIQUE REFERENCES products(id) ON DELETE CASCADE,

    -- Informatii despre stock
    quantity INT NOT NULL DEFAULT 0,

```

```

        last_updated TIMESTAMP NOT NULL DEFAULT now()
    );

]

```

5.4 Recommendations

```

-- Tabela ce face asocierea dintre un user si raspunsurile la chestionar
-- 0 sesiune de recomandari = un nou raspuns inregistrat

```

```

CREATE TABLE recommendation_sessions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- cheie primara

    -- Asocierea recomandarilor cu utilizatorul
    -- Cand utilizatorul este sters se sterg si toate informatiile
    -- asociate lui din aceasta tabela
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    -- Asocierea recomandarilor cu raspunsurile la chestionar
    profile_id UUID NOT NULL REFERENCES user_profiles(id),

    created_at TIMESTAMP NOT NULL DEFAULT now(),
    user_level_at_time current_level_type NOT NULL
);

```

```

-- Asocierea recomandarilor cu tabela sports

```

```

CREATE TABLE recommendation_sports (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- cheie primara

    -- Asocierea de tip one to many cu tabela recommendation_sessions
    -- Daca sesiunea este stearsa se vor sterge si toate sporturile din acea sesiune
    session_id UUID NOT NULL REFERENCES recommendation_sessions(id) ON DELETE CASCADE,

    -- Asocierea cu tabela sports
    -- 0 singura recomandare genereaza mai multe sporturi
    sport_id UUID NOT NULL REFERENCES sports(id),
    compatibility_score NUMERIC(5,2) NOT NULL,

```

```

    rank INT NOT NULL,

    -- In cadrul unei sesiuni nu poate fi recomandat de mai multe ori
    -- acelasi sport
    UNIQUE (session_id, sport_id)
);

-- Asocierea recomandarilor cu tabela products
CREATE TABLE recommendation_products (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- cheie primara

    -- Asocierea de tip one to many cu tabela recommendation_sessions
    -- Daca sesiunea este stearsa se vor sterge si toate sporturile din acea sesiune
    session_id UUID NOT NULL REFERENCES recommendation_sessions(id) ON DELETE CASCADE,

    -- Asocierea cu tabela products
    -- O singura recomandare genereaza mai multe produse
    product_id UUID NOT NULL REFERENCES products(id),

    -- Asocierea cu tabela sports
    -- Pentru fiecare sport recomandat sunt recomandate si produse asociate
    -- acelui sport
    sport_id UUID NOT NULL REFERENCES sports(id),
    reason VARCHAR(255),

    -- In cadrul unei sesiuni nu poate fi recomandat de mai multe ori
    -- acelasi produs
    UNIQUE (session_id, product_id)
);

```

5.5 Orders

```

CREATE TYPE order_status_type AS ENUM (
    'PENDING',
    'CONFIRMED',

```

```

        'SHIPPED',
        'DELIVERED',
        'CANCELLED'
    );

-- Informatii despre comenzi
CREATE TABLE orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- cheie primara

    -- Asocierea comenzii cu tabela users
    user_id UUID NOT NULL REFERENCES users(id),

    -- Asocierea comenzii cu sesiunea de recomandari
    -- Folositoare pentru statistici privind recomandarile
    -- Nu este obligatorie, asa ca daca recomandarea este stearsa,
    -- pot exista comenzi ce nu sunt asociate la o recomandare
    session_id UUID REFERENCES recommendation_sessions(id) ON DELETE SET NULL,

    status order_status_type NOT NULL DEFAULT 'PENDING',
    total_amount NUMERIC(10, 2) NOT NULL,

    -- Exista si in tabela users dar isi are locul si aici deoarece se doreste
    -- pastrarea istoricului la care au fost distribuite comenzile
    shipping_address TEXT NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT now(),
    updated_at TIMESTAMP NOT NULL DEFAULT now()
);

-- Informatii despre produsele unei comenzi
CREATE TABLE order_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- cheie primara

    -- Asocierea cu tabela orders
    order_id UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE,

```

```

-- Asocierea cu tabela products
product_id UUID NOT NULL REFERENCES products(id),

-- Informatii legate de cantitatea produsului
quantity INT NOT NULL,

-- Se tine cont de istoricul produsului (pretul la care a fost cumparat)
-- Nu este afectat de schimbarile curente de pret a produsului
unit_price NUMERIC(10,2) NOT NULL,

-- Intr-o comanda nu se poate introduce de doua ori acela
UNIQUE (order_id, product_id)
);

```

6 Prezentarea Aplicației

6.1 Dashboard

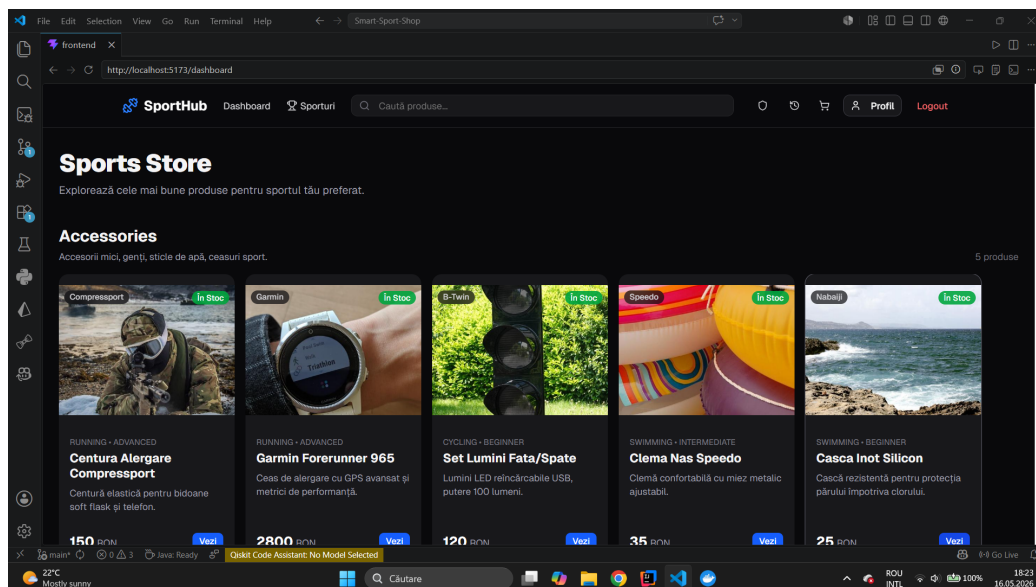
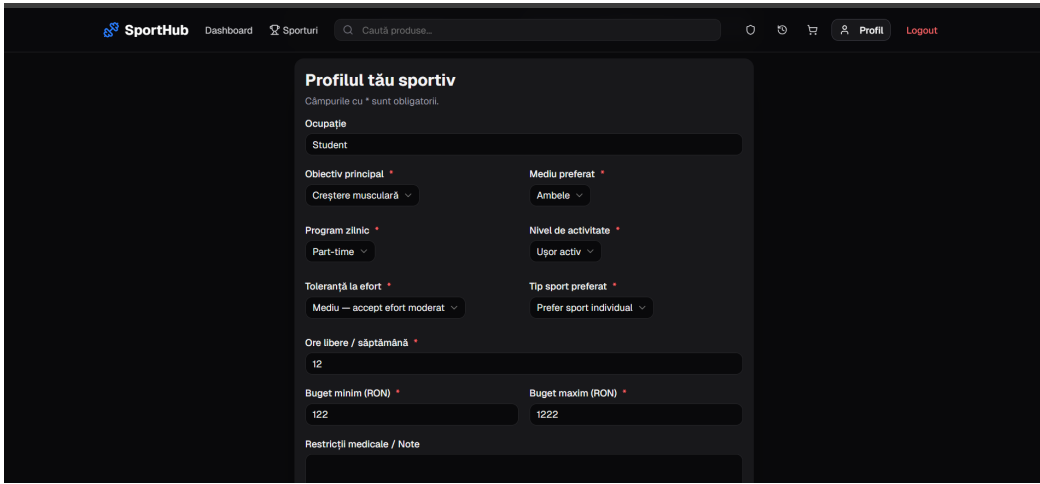


Figure 2: Pagina principală a aplicației

Pagina principală oferă utilizatorului o privire de ansamblu asupra aplicației, prezentând produsele disponibile și opțiunea de a accesa chestionarul de recomandări.

6.2 Chestionar



SportHub Dashboard Sporturi [Profil](#) [Logout](#)

Profilul tău sportiv

Câmpurile cu * sunt obligatorii.

Ocupație

Obiectiv principal *

Mediu preferat *

Program zilnic *

Nivel de activitate *

Toleranță la efort *

Tip sport preferat *

Ore libere / săptămână *

Buget minim (RON) *

Buget maxim (RON) *

Restricții medicale / Note

Figure 3: Chestionarul de evaluare a profilului utilizatorului

Utilizatorul completează chestionarul cu informații despre stilul de viață, obiective și preferințe. Pe baza răspunsurilor, algoritmul de recomandare calculează scoruri de compatibilitate pentru fiecare sport disponibil.

6.3 Recomandări

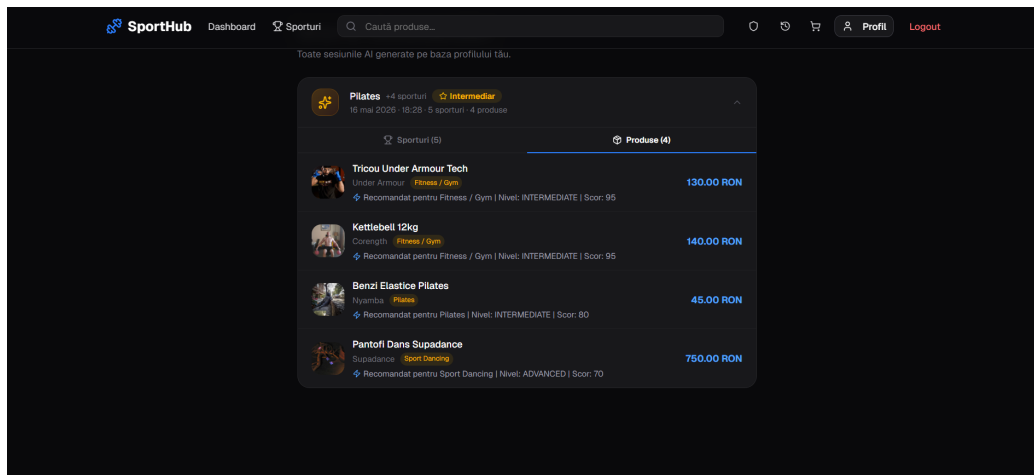


Figure 4: Pagina de recomandări generate de sistem

După completarea chestionarului, sistemul prezintă sporturile recomandate ordered după scorul de compatibilitate, împreună cu produsele asociate nivelului și bugetului utilizatorului.

6.4 Pagina de Comenzi

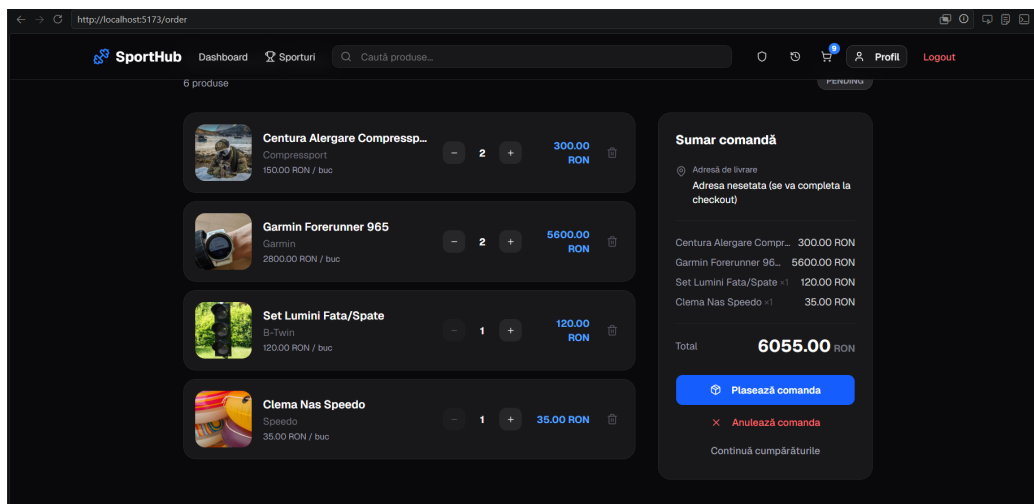


Figure 5: Istoricul comenzilor utilizatorului

Utilizatorul poate vizualiza istoricul complet al comenzilor plasate, inclusiv statusul fiecărei comenzi și produsele achiziționate.

6.5 Panou de Administrare

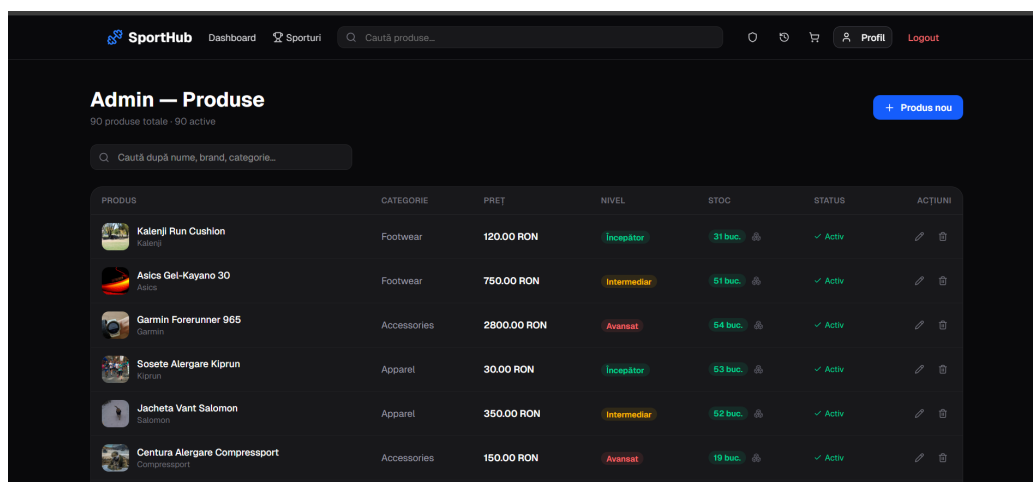


Figure 6: Panoul de administrare pentru gestiunea produselor și stocurilor

Panoul de administrare permite gestionarea produselor, categoriilor și stocurilor. De aici se pot executa operații de tip CRUD pentru produse și se poate actualiza cantitatea disponibilă în stoc.

7 Algoritmul de Recomandare

Algoritmul este implementat integral în PL/pgSQL sub forma unui lanț de funcții care se apelează în cascadă, orchestrat de funcția principală `generate_recommendations`.

7.1 Determinarea nivelului utilizatorului

Funcția `evaluate_user_level` calculează un scor pe baza a trei criterii din profil: nivelul de activitate (0–4 puncte), toleranța la efort (0–4 puncte) și orele libere săptămânal (0–2 puncte). Scorul final determină nivelul: sub 4 → *BEGINNER*, 4–6 → *INTERMEDIATE*, peste 7 → *ADVANCED*.

7.2 Scorurile de compatibilitate pentru sporturi

Funcția `calculate_sport_scores` sumează ponderile criteriilor îndeplinite din `sport_criteria` și aplică un factor de 1.2 dacă preferința pentru sport individual/echipă se potrivește, respectiv 0.8 în caz contrar. Sunt returnate sporturile active ordonate descrescător după scor.

7.3 Selectarea produselor

Funcția `calculate_product_recommendations` punctează fiecare produs după patru criterii: potrivirea nivelului (max 40p), încadrarea în buget (max 30p), rangul sportului asociat (max 20p) și disponibilitatea în stoc (max 10p). Din produsele candidate se selectează cel mai bun per categorie folosind `DISTINCT ON`, evitând redundanța în recomandări.

7.4 Triggerul de declanșare

Un trigger `AFTER INSERT` pe `user_profiles` apelează automat întregul algoritm la fiecare completare a chestionarului. Se folosește `INSERT` și nu `UPDATE` deoarece fiecare completare creează un rând nou, păstrând istoricul sesiunilor anterioare.

```
CREATE TRIGGER trg_generate_recommendations
  AFTER INSERT ON user_profiles
  FOR EACH ROW
  EXECUTE FUNCTION trg_generate_recommendations_fn();
```

References

- [1] Facultatea de Informatică, Universitatea Alexandru Ioan Cuza Iași. *Curs Baze de Date — Cursurile 5 și 6*. <https://edu.info.uaic.ro/baze-de-date/ro/index.html>
- [2] PostgreSQL Global Development Group. *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>